Reinforcement Learning

Third Assignment

Federica Lavini

Matricola: 1941986

Contents

1 Theory **3**
1.1 REINFORCE Algorithm: Weight Update . . . 3
1.1.1 Algorithm and Policy Parameterization . . . 3
1.1.2 Discounted Return and Log-Policy Gradient . . . 3
1.1.3 Problem Solution . . . 3

2 Practice **5**
2.1 PPO algorithm . . . 5
2.2 Implementation . . . 5
2.3 Results . . . 8

3 Collaborations **9**

1 Theory

1.1 REINFORCE Algorithm: Weight Update

In this first exercise is about using the **REINFORCE algorithm**.The objective is to compute the update of the policy parameters after observing a single episode, using a stochastic policy with a linear function approximator.

1.1.1 Algorithm and Policy Parameterization

REINFORCE is a policy gradient algorithm that directly updates the parameters of a stochastic policy in order to maximize the expected return. The parameter update rule is:

$$w \leftarrow w + \alpha \sum_{t=0}^{T-1} G_t \nabla_w \log \pi(a_t \mid s_t),$$

where α is the learning rate, $\pi(a_t \mid s_t)$ is the stochastic policy, and G_t is the discounted return starting from time step t. The update is applied after a full episode by summing the contributions of all time steps.

The policy is parameterized using a linear function approximator followed by a sigmoid activation function:

$$\pi(a = 1 \mid s) = \sigma(w^\top x(s)), \quad \sigma(t) = \frac{1}{1 + e^{-t}}.$$

This defines a Bernoulli distribution over two possible actions $a \in \{0, 1\}$. The initial weight vector is:

$$w = \begin{bmatrix} 0.8 \\ 1 \end{bmatrix}.$$

1.1.2 Discounted Return and Log-Policy Gradient

The discounted return from time step t is defined as:

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1},$$

where $\gamma \in [0, 1]$ is the discount factor. In this example, $\gamma = 0.9$.

For a binary logistic policy, the gradient of the log-probability with respect to the policy parameters is given by:

$$\nabla_w \log \pi(a \mid s) = \begin{cases} (1 - \pi(1 \mid s))\, x(s), & a = 1, \\ -\pi(1 \mid s)\, x(s), & a = 0. \end{cases}$$

This expression is used directly to compute the REINFORCE update for each time step.

1.1.3 Problem Solution

The learning rate is $\alpha = 0.1$ and the discount factor is $\gamma = 0.9$. The observed episode is:

t	$x(s_t)$	a_t	r_{t+1}
0	$[1, 0]^\top$	0	0
1	$[1, 0]^\top$	1	1
2	$[0, 1]^\top$	–	–

Using the definition of discounted return we compute returns:

$$G_0 = r_1 + \gamma r_2 = 0 + 0.9 \cdot 1 = 0.9,$$

$$G_1 = r_2 = 1.$$

Then we compute the policy probabilities at time steps $t = 0$ and $t = 1$, the state representation is the same. Therefore, the policy probability is identical in both cases.

$$w^\top x(s_0) = [0.8, 1] \cdot [1, 0]^\top = 0.8,$$

$$\pi(1 \mid s_0) = \sigma(0.8) \approx 0.690, \quad \pi(0 \mid s_0) \approx 0.310.$$

We continue with the Log-Policy gradients:

- For $t = 0$, the taken action is $a_0 = 0$:

$$\nabla_w \log \pi(0 \mid s_0) = -0.690 \cdot [1, 0]^\top = [-0.690, 0]^\top.$$

- For $t = 1$, the taken action is $a_1 = 1$:

$$\nabla_w \log \pi(1 \mid s_1) = (1 - 0.690) \cdot [1, 0]^\top = [0.310, 0]^\top.$$

The individual contributions to the weight update are:

$$\Delta w_0 = \alpha G_0 \nabla_w \log \pi(a_0 \mid s_0) = 0.1 \cdot 0.9 \cdot [-0.690, 0]^\top = [-0.0621, 0]^\top,$$

$$\Delta w_1 = \alpha G_1 \nabla_w \log \pi(a_1 \mid s_1) = 0.1 \cdot 1 \cdot [0.310, 0]^\top = [0.0310, 0]^\top.$$

Summing both contributions gives the updated weights:

$$w_{\text{new}} = \begin{bmatrix} 0.8 \\ 1 \end{bmatrix} + \begin{bmatrix} -0.0621 \\ 0 \end{bmatrix} + \begin{bmatrix} 0.0310 \\ 0 \end{bmatrix} = \begin{bmatrix} 0.7689 \\ 1 \end{bmatrix}.$$

After one episode, the updated weight vector is:

$$\boxed{w_{\text{new}} \approx \begin{bmatrix} 0.769 \\ 1 \end{bmatrix}}$$

Only the first component of the weight vector is updated, since the second state feature is zero for all visited states. The negative contribution from $a_0 = 0$ slightly decreased w_1, while $a_1 = 1$ increased it. This example illustrates a straightforward application of the REINFORCE algorithm with a linear logistic policy and Monte Carlo returns.

2 Practice

For the practical exercise, I decided to implement the Proximal Policy Optimization, PPO, algorithm for the CarRacing-v2 environment. I chose PPO because it represents a good compromise between implementation simplicity, training stability, and strong empirical performance, especially in continuous control environments.

2.1 PPO algorithm

PPO belongs to the family of actor–critic methods and mixes two actions: collecting experience by interacting with the environment and optimizing the policy using mini-batch gradient updates. The main idea behind PPO is to limit how much the policy can change at each update, using a clipped objective function to avoid overly large and unstable updates. Unlike TRPO, PPO avoids complex constrained optimization based on KL-divergence, making it more practical while retaining comparable empirical performance. For this exercise, PPO is particularly suitable due to its robustness and effectiveness in environments with high-dimensional observations and continuous or discrete action spaces, such as CarRacing-v2.

Algorithm 1 Reinforcement Learning with Proximal Policy Optimization (PPO)

for each episode **do**
  Initialize memory $\mathcal{M}$
  for each timestep $t = 0, \ldots, T$ **do**
    Select action $a_t \sim \pi_\theta(a_t \mid s_t)$
    Execute action a_t in the environment
    Observe reward r_t, next state s_{t+1}, and done flag $done_t$
    Store $(s_t, a_t, r_t, \log \pi_{\theta_{\text{old}}}(a_t \mid s_t), done_t)$ in $\mathcal{M}$
  end for
  Compute state values $V_{\theta_{\text{old}}}(s_t)$ for all $s_t \in \mathcal{M}$
  Compute returns G_t and advantages $\hat{A}_t$ using GAE
  Normalize advantages $\hat{A}_t$
  for $k = 1$ to K (optimization epochs) **do**
    Sample mini-batches from $\mathcal{M}$
    Compute probability ratio:

$$r_t(\theta) = \exp\left(\log \pi_\theta(a_t \mid s_t) - \log \pi_{\theta_{\text{old}}}(a_t \mid s_t)\right)$$

    Compute clipped surrogate objective:

$$L^{\text{CLIP}} = \min\left(r_t(\theta)\hat{A}_t, \text{clip}\left(r_t(\theta), 1-\epsilon, 1+\epsilon\right)\hat{A}_t\right)$$

    Compute value function loss:

$$L^{\text{VF}} = \text{MSE}(V_\theta(s_t), G_t)$$

    Compute entropy bonus to encourage exploration
    Compute total loss:

$$L = -L^{\text{CLIP}} + c_1 L^{\text{VF}} - c_2 \mathcal{H}(\pi_\theta)$$

    Perform gradient clipping
    Update parameters θ using Adam optimizer
  end for
end for

2.2 Implementation

This section describes the main design choices and is going deep to the implementation.

Nel codice ho scelto la versione disceta di CarRacing, continuous = False, per una maggiore stabilità iniziale dell'implementazione ma comunuqe rimane compatibile con la versione continua.

The agent is implemented as an actor-critic model with a shered convolutional encoder and two separate output heads: one for the actor, the policy and one for the critic, the value function. Meaningful features from the RGB images provided by the enviroment are extrated by the shered layers, as shown in Listing 1. These are used by the actor and the critic, improving computational efficiency and ensuring a consistent state representation. The use of strided convolutions progressively reduces the spatial resolution while preserving relevant information.

Listing 1: Shared convolutional encoder used by actor and critic

```
self.shared = nn.Sequential(
    nn.Conv2d(3, 32, kernel_size=8, stride=4),
    nn.ReLU(),
    nn.Conv2d(32, 64, kernel_size=4, stride=2),
    nn.ReLU(),
    nn.Conv2d(64, 64, kernel_size=3, stride=1),
    nn.ReLU(),
    nn.Flatten(),
    nn.Linear(64 * 8 * 8, 256),
    nn.ReLU()
)
```

The main hyperparameters used for the PPO implementation are:

Listing 2: PPO hyperparameters

```
self.clip_epsilon = 0.2
self.entropy_coef = 0.01
self.value_coef = 0.5
self.learning_rate = 1e-4
self.optimizer = torch.optim.Adam(self.parameters(), lr=self.learning_rate)

self.gamma = 0.99
self.lambda_gae = 0.95
self.max_episodes = 7500
self.ppo_update_epochs = 5
self.batch_size = 64
self.max_timesteps = self.batch_size * 4
self.skip_frames = 4
```

A value of 0.2 for the clipping parameter is commonly used in PPO implementations and represents a good trade-off between learning speed and stability. The entropy coefficient weights the entropy bonus added to the loss function. The entropy coefficient encourages exploration, a relatively small value is chosen to promote it during the initial training phase. The value function coefficient is set at 0.5 to ensure that the critic learns accurate value estimates without dominating the overall optimization process. The lean rate of 10^{-4} is commonly used in deep reinforcement learning with convulational networks to reduce oscillations and unstable training. As optimizer I chose Adam. The discount factor encourages the agent to consider long-term rewards, very important here where success depends on sustained driving performance. The GAE is 0.95 provides a good compromise between low variance and limited bias. The total number of training episodes is set to 7500 to allow sufficient exploration and convergence in a complex environment. Each batch of collected data is reused for multiple optimization epochs. Using 5 PPO update epochs improves sample efficiency without overfit. A batch size of 64 provides reasonably stable gradient estimates while remaining computationally efficient and memory-friendly. The maximum number of timesteps per rollout is set proportional to the batch size. Using a skip value of 4 exploits the temporal redundancy in the CarRacing environment while preserving sufficient control resolution.

After feature estraction, the network splits into two heads: the actor head and the critic head. The first one produces the parameters of the policy. In the discrete case, it outputs action logits that are converted into action probabilities using a softmax function. In the continuous case, it outputs the mean of a Gaussian distribution, while the standard deviation is learned separately. The second head has as outputs a single scalar value representing the state-value function V(s), which estimates how good the current state is.

Inside the forward() method all observations are processed directely: images are converted from HWC to NCHW format, pixel values are normalized to the range [0,1], both single states and batches

are supported.

THen, since PPO relies on a clipped surrogate objective, the total loss function is composed of three terms:

$$\mathcal{L} = -\mathcal{L}_{\text{CLIP}} + c_1 \mathcal{L}_{\text{VF}} - c_2 \mathcal{H}, \tag{1}$$

where $\mathcal{L}_{\text{CLIP}}$ is the policy loss, $\mathcal{L}_{\text{VF}}$ is the value function loss, and $\mathcal{H}$ is the entropy bonus.

The clipped surrogate objective is defined as:

$$\mathcal{L}_{\text{CLIP}} = \mathbb{E}_t \left[\min \Big(r_t(\theta)\, \hat{A}_t,\ \text{clip}\big(r_t(\theta), 1-\epsilon, 1+\epsilon\big)\, \hat{A}_t \Big) \right], \tag{2}$$

where $r_t(\theta)$ is the probability ratio between the new and old policies:

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}. \tag{3}$$

This objective prevent excessively large changes that could destabilize learning.

The value loss is computed as the mean squared error between the critic's estimate and the target return:

$$\mathcal{L}_{\text{VF}} = \left(V_\theta(s_t) - G_t \right)^2. \tag{4}$$

An entropy bonus $\mathcal{H}$ is added to the loss function to encourage exploration and to prevent the policy from becoming deterministic too early.

To estimate the advantage function, GAE is used. Inside the `compute_advantages()` method, the advantage at timestep t is computed as:

$$\hat{A}_t = \delta_t + \gamma \lambda \hat{A}_{t+1} (1 - \text{done}), \tag{5}$$

where the temporal-difference error δ_t is defined as:

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t). \tag{6}$$

This formulation provides a controlled trade-off between bias and variance.

To avoid excessively large or small values influencing training disproportionately, advantages are normalized before being used in the PPO update.

The target returns used for the critic are computed as:

$$G_t = \hat{A}_t + V(s_t). \tag{7}$$

All convolutional and linear layers are initialized using orthogonal initialization, that helps to preserve the preserve the scale of activations across layers and reduce vanishing and exploding gradients.

In the continuous action setting, the standard deviation of the action distribution is obtained using the Softplus activation function:

$$\text{action_std} = \text{Softplus}(\text{actor_log_std}). \tag{8}$$

This guarantees that the standard deviation is always positive and provides smooth and good gradients.

In the training loop:

- trajectories are collected in an on-policy manner,
- PPO updates are performed for multiple optimization epochs,
- the collected data is shuffled and split into mini-batches of size 64.

Gradient clipping is applied during backpropagation to prevent excessively large parameter updates, significantly improving training stability.

To further stabilize learning:

- rewards are clipped to a maximum value of 1.0,
- frame skipping is applied with skip_frames = 4.

Instead of using only the reward from the last transition, the average reward across the skipped frames is computed. This reduces computational cost while preserving relevant information, since consecutive frames in the CarRacing environment are highly similar.

When using continuous actions, samples drawn from the policy distribution may fall outside the valid action range of the environment. To prevent invalid actions, sampled values are clipped to lie within the bounds defined by:

$$\text{action} \in [\text{action_space.low}, \text{action_space.high}]. \tag{9}$$

2.3 Results

At the beginning of training, the average reward is around -40, indicating that the agent behaves almost randomly.

After approximately 500 episodes, a clear improvement can be observed. Before this point, the policy is unstable and the car frequently leaves the track.

With further training, the agent learns to drive reasonably well on different types of curves and often reaches the end of the track, achieving high reward values. However, the learned behavior is not always optimal. In some situations, especially in tight or rapidly alternating curves, the car may cut parts of the track or perform a full rotation (360 degrees) before continuing. In several cases, the agent is able to recover and return to the track, while in others it leaves the road and fails to recover.

One important limitation of the environment is related to the observations. When the car moves completely onto the grass, no track pixels are visible in the input image. In these situations, the agent lacks sufficient visual information to reorient itself, making recovery impossible. This limitation is inherent to the observation space and not due to a flaw in the learning algorithm.

When evaluating the trained model using `main.py`, the average reward ranges between 700 and 850, which indicates good overall performance. It should be noted that rewards are clipped, as discussed in previous sections, and therefore the reported values must be interpreted accordingly.

As shown in the code, the implementation supports both discrete and continuous action spaces through the `self.continuous` flag. However, training with continuous actions often leads to numerical instability. During debugging, it was observed that after several hundred episodes, gradients and network parameters occasionally become NaN, causing invalid outputs during the forward pass. Despite multiple attempts to address this issue, it could not be fully resolved. For this reason, this report focuses exclusively on results obtained using the discrete action space, which proved to be more stable and reliable.

In conclusion, the trained agent achieves high rewards and demonstrates competent driving behavior, but some limitations remain. These include imperfect handling of sharp curves and occasional unrealistic maneuvers. Possible future improvements include improving stability in the continuous action setting and introducing parallelized training. A multi-actor approach, where multiple agents collect experience simultaneously, could improve both learning efficiency and generalization. However, this was not implemented due to restrictions on importing additional libraries.

3 Collaborations

I discussed the assignment with Flaminia Lanzellotti, matricola number 2201485.